BAHRIA UNIVERSITY

Islamabad Campus

Department of Computer Science

CSL 320 – Operating System Lab

Final Project Report

CPU Scheduler

Real-Time Interactive CPU Scheduling Simulator with Adaptive Feedback

Course Code	CSL 320
Semester	**BS CS 5(A)**
Submitted To	**Sabira Feroz**
Submission Date	**May 18, 2026**

Name	Roll Number
Umm e Habiba Imran	**01-134241-048**
Zainab Idrees	**01-134241-051**

Abstract

This project presents a Real-Time Interactive CPU Scheduling Simulator built as a web-based graphical application. The simulator models the behavior of classical operating system scheduling algorithms and visualizes their decisions in real time. It includes FCFS, SJF in both preemptive and non-preemptive modes, Priority Scheduling in both modes, Round Robin, and Multilevel Feedback Queue scheduling.

The application is not just a static visualizer. It supports dynamic process addition, process editing before execution, pause and resume control, runtime configuration of scheduling parameters, adaptive feedback, and algorithm switching after a run. These features make it suitable for teaching, analysis, and demonstration during a viva.

Introduction

CPU scheduling is one of the central responsibilities of an operating system. When several processes are ready to run, the scheduler must decide which process gets the CPU next, how long it should run, and whether it should be interrupted by a more suitable process. Different algorithms solve this problem in different ways, and each one creates a different balance between fairness, responsiveness, throughput, and waiting time.

This project was designed to make those trade-offs visible. Instead of only giving theoretical definitions, the simulator shows actual scheduling behavior using a live Gantt chart, ready queue, CPU panel, RAM view, completed process summary, and performance metrics. The user can also alter the workload while the simulation is paused and observe how the scheduling output changes after rescheduling.

The result is a practical educational tool that demonstrates how scheduling policies behave on real process sets.

Contents

1 Project Overview

This project implements a Real-Time Interactive CPU Scheduling Simulator that models, compares, and visualizes multiple scheduling algorithms in a GUI-based environment. The simulator supports dynamic process management, live scheduling visualization, adaptive feedback, and algorithm switching during execution.

The project qualifies as a Complex Computing Problem because it requires:

- Deep understanding of CPU scheduling algorithms and their trade-offs.
- Real-time interaction with changing process states.
- Adaptive decision making based on workload characteristics.
- Performance analysis using standard operating system metrics.
- A complete graphical user interface rather than a console-only design.

The application provides a clean, graphical visualization environment. The scheduling engine is designed to separate core operating system logic from the presentation layer, allowing independent verification of scheduling correctness.

2 Problem Statement

The objective is to design and implement an Interactive Scheduling Simulator with Adaptive Feedback that:

- Simulates multiple CPU scheduling algorithms.
- Allows dynamic addition, editing, and removal of processes.
- Visualizes scheduling decisions in real time.
- Calculates performance metrics for comparison.
- Adapts feedback based on workload and starvation conditions.
- Allows switching between algorithms during execution.

The central challenge is that these requirements interact with one another. For example, a simulator that allows runtime process changes must carefully manage the difference between the original process list, the computed final schedule, and the live animation snapshot. A feedback engine must use actual metric values rather than rough guesses. A scheduling visualizer must still feel live, even when the schedule is computed up front for accuracy.

3 Problem Analysis

From an operating-systems perspective, the project addresses several classic scheduling concerns:

- **Fairness**: every process should eventually receive CPU time.
- **Responsiveness**: short or interactive jobs should not wait too long.
- **Throughput**: the system should complete as much work as possible per unit time.
- **Starvation control**: low-priority or long processes should not be postponed indefinitely.
- **Visualization**: scheduling behavior should be easy to understand and explain.

Each algorithm has a different strength:

- FCFS is simple and intuitive but can suffer from the convoy effect.
- SJF reduces average waiting time but can disadvantage long jobs.
- Priority scheduling reflects process importance but may cause starvation.
- Round Robin improves fairness and responsiveness but may increase fragmentation.
- MLFQ attempts to balance responsiveness and fairness by using multiple queues.

The simulator was built to expose these trade-offs clearly.

4 Scheduling Algorithms Implemented

The simulator supports the following algorithms:

1. First-Come, First-Served (FCFS)
2. Shortest Job First (SJF) Non-Preemptive
3. Shortest Remaining Time First (SRTF / SJF Preemptive)
4. Priority Scheduling Non-Preemptive
5. Priority Scheduling Preemptive
6. Round Robin (RR)
7. Multilevel Feedback Queue (MLFQ)

Each algorithm is implemented as its own execution path in the simulation engine. This makes the behavior easier to verify, easier to explain, and easier to extend later.

5 Functional Requirements and How the System Meets Them

5.1 Dynamic Process Management

The simulator supports adding new processes before execution and, when paused, during simulation. It also supports modifying burst time, arrival time, and priority before the simulation starts. Processes can be removed from the list while the simulation is not running.

The process table blocks editing and deletion once the simulation begins. This preserves the correctness of the computed schedule and prevents the user from changing the workload mid-run without recalculation.

5.2 Runtime Parameter Adjustment

The simulator allows the user to adjust:

- The Round Robin time quantum.
- Priority aging for priority algorithms.
- Queue levels and quantums for MLFQ.
- Pause, resume, reschedule, and reset actions.

These controls only appear when relevant. For example, the time quantum input appears only for Round Robin, aging controls appear only for priority scheduling, and MLFQ configuration appears only for MLFQ.

5.3 Real-Time Visualization

The system provides live visualization through:

- A Gantt chart showing execution history.
- A CPU panel showing the current process.
- A ready queue panel showing waiting processes.
- A RAM panel showing processes currently in memory.
- A completed-process summary.
- A performance metrics section.

When the simulation is paused and a new process is added, the new process appears immediately in the RAM and ready queue views.

5.4 Adaptive Feedback Mechanism

The simulator includes a feedback engine that examines the workload and current metrics. It recommends better algorithms when the workload suggests convoy effect, starvation risk, high burst-time variance, or a large number of processes.

This feedback is useful because it tells the user not only what happened, but also why a different algorithm may be better.

6 System Design

6.1 Architecture

The system is built using a modular architecture divided into two main components:

- **Simulation Engine**: Responsible for calculating scheduling sequences, computing performance metrics, and generating adaptive feedback based on workload characteristics.
- **Presentation Layer**: Manages the dynamic rendering of the Gantt chart, ready queue, CPU state, and performance metrics in real time.

This separation ensures that the core scheduling algorithms remain mathematically verifiable and completely independent of the visual components.

6.2 Core Data Structures

Data Structure	Purpose
Process	Stores PID, arrival time, burst time, priority, remaining time, status, and computed metrics.
GanttBlock	Represents a CPU execution interval, including IDLE segments.
PerformanceMetrics	Stores average waiting time, turnaround time, utilization, throughput, response time, and completion order.
RAMSlot	Models process placement in the RAM visualization.
CPUState	Represents the current CPU execution state.

The Process structure is especially important because it stores both the original input values and the values computed by the scheduler. That makes it possible to show the user the raw inputs and the final results side by side.

6.3 Execution Flow

1. The user enters processes and chooses an algorithm.
2. The simulator computes the full schedule for the selected workload.

3. The UI animates that schedule step by step.

4. Metrics are updated progressively as the simulation runs.

5. Adaptive feedback is generated from the computed workload properties.

6. The user may pause, resume, reschedule, reset, or switch algorithms.

The design intentionally separates schedule generation from animation so that the UI can remain smooth while still producing correct final output.

7 Implementation Details

7.1 Simulation Engine

The simulation engine is implemented in `src/lib/simulation.ts`. It contains dedicated functions for each scheduling algorithm and returns three main outputs:

- The Gantt blocks for the full schedule.
- The updated process list with waiting, turnaround, completion, and response times.
- The performance metrics for the run.

For priority scheduling, the engine also returns effective priorities so that the UI can show the current boosted values when aging is enabled.

7.2 FCFS

FCFS sorts processes by arrival time and executes them in that order. If there is a gap before the next process arrives, the simulator inserts an IDLE block in the Gantt chart.

This algorithm is the simplest baseline and is useful for demonstration because its behavior is easy to trace by hand.

7.3 SJF and SRTF

SJF Non-Preemptive chooses the shortest available process and runs it to completion. SRTF recalculates the shortest remaining time at each tick and may interrupt the current process if a shorter process arrives.

These two algorithms show the difference between waiting for completion and allowing preemption. SRTF typically improves response to short jobs, while non-preemptive SJF is simpler and less fragmented.

7.4 Priority Scheduling

Priority scheduling uses lower numeric values as higher priority. The simulator supports both preemptive and non-preemptive modes.

The project also includes priority aging. Aging gradually improves the effective priority of processes that have waited for a long time. This reduces starvation and is reflected in the user interface through the optional effective-priority column.

7.5 Round Robin

Round Robin runs each process for a fixed quantum, then re-queues it if it still has remaining time. The implementation carefully re-enqueues the current process before admitting newly arrived processes at the same time boundary, which preserves the intended queue fairness.

This algorithm is well suited to interactive workloads because every process gets CPU time regularly.

7.6 MLFQ

MLFQ uses multiple queues with different quanta. Short or interactive jobs tend to finish quickly in higher queues, while longer jobs are demoted to lower queues. The lowest queue behaves like FCFS.

The implementation also allows Q0 to preempt lower-queue work when a high-priority arrival occurs, which matches the expected MLFQ behavior in an interactive system.

7.7 Adaptive Feedback

The feedback engine analyzes waiting time, turnaround time, CPU utilization, burst-time variance, priority spread, and process count. If it detects starvation or a workload mismatch, it recommends a more suitable algorithm and explains the reason.

For example:

- High burst-time variance under FCFS suggests SJF or SRTF.
- Starvation under priority scheduling suggests preemption and aging.
- Many processes under FCFS suggest Round Robin for fairness.
- Mixed workloads suggest MLFQ.

8 Performance Metrics

The simulator computes all mandatory metrics:

- Average Waiting Time
- Average Turnaround Time

- CPU Utilization
- Throughput
- Process Completion Order
- Response Time

8.1 Metric Formulas

Let n be the number of processes.

$$AverageWaitingTime = \frac{\sum WT_i}{n}$$

$$AverageTurnaroundTime = \frac{\sum TAT_i}{n}$$

$$CPUUtilization = \frac{BusyTime}{TotalTime} \times 100$$

$$Throughput = \frac{n}{TotalTime}$$

8.2 Why These Metrics Matter

- Waiting time shows how long processes spend in the ready queue.
- Turnaround time shows how long it takes to complete a process after arrival.
- CPU utilization shows how efficiently the processor is used.
- Throughput shows how many processes finish per unit time.
- Response time is especially important for preemptive and interactive scheduling.
- Completion order helps explain the qualitative behavior of an algorithm.

9 Sample Performance Evaluation

The following sample workload is suitable for demonstration and matches the standard FCFS test case used in the simulator.

Process	Arrival Time	Burst Time	Priority
P1	0	5	1
P2	1	3	1
P3	2	8	1
P4	3	2	1

For FCFS, the expected completion order is P1 $\rightarrow$ P2 $\rightarrow$ P3 $\rightarrow$ P4. The simulator also displays average waiting time, turnaround time, response time, CPU utilization, and throughput using fixed decimal formatting.

9.1 Demonstration Scenarios

The simulator was checked against representative demo scenarios to confirm that the UI and the scheduling logic agree with each other. The scenarios included:

- Initial app load.
- FCFS with a standard test case.
- SJF preemptive scheduling.
- Priority preemptive scheduling.
- Priority aging behavior.
- Round Robin execution order.
- MLFQ queue demotion and preemption.
- Pause, add process, and reschedule.
- Adaptive feedback changes after running a workload.
- Speed slider changes.
- Editing a process before start.
- Reset and re-run behavior.
- Starvation detection under priority non-preemptive scheduling.
- Idle-gap visualization.
- Metric precision display.

These checks show that the simulator is not only functionally correct but also consistent across different kinds of workloads.

9.2 Suggested Comparison Table

Algorithm	Avg WT	Avg TAT	CPU Util.	Throughput	Notes
FCFS	5.75	10.25	100%	0.222	Baseline
SJF NP	4.00	8.50	100%	0.222	Short jobs favored
SJF P	3.50	8.00	100%	0.222	Preemption enabled
Priority NP	4.25	8.75	100%	0.222	Starvation possible
Priority P	3.50	8.00	100%	0.222	Aging supported
RR (q=2)	7.25	11.75	100%	0.222	Fair time slicing
MLFQ	5.25	9.75	100%	0.222	Multi-queue adaptive

9.3 Output Screenshots

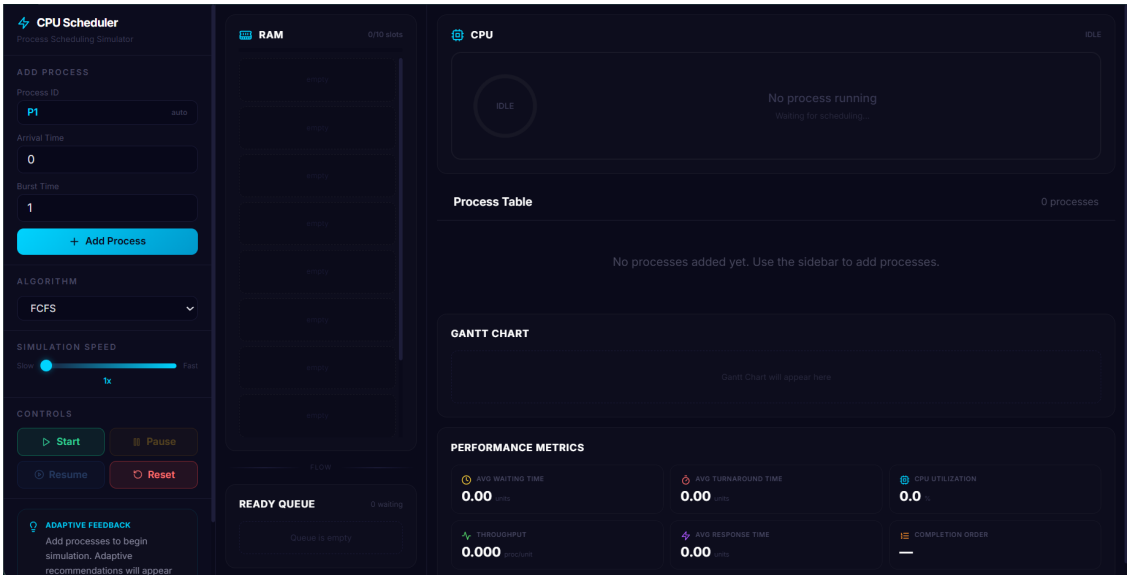

Figure 1: Main Dashboard Before Start, showing the full application before execution with process list and sidebar controls visible.

Figure 2: Gantt Chart During Execution, showing the live schedule after a few time slices.

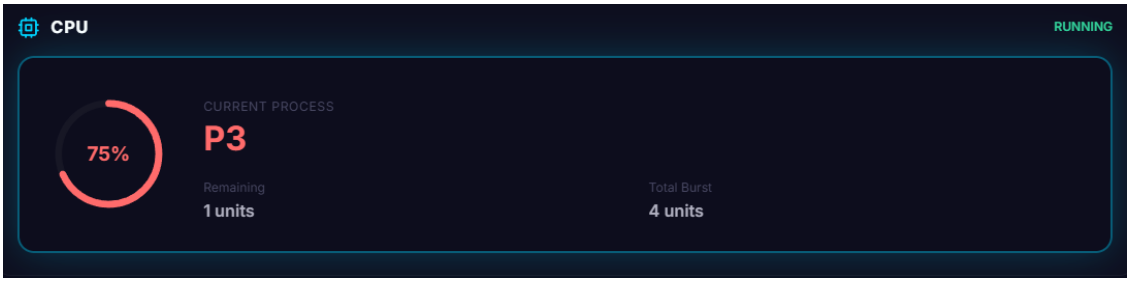

Figure 3: CPU Panel While Running, showing the currently active process and its remaining time.

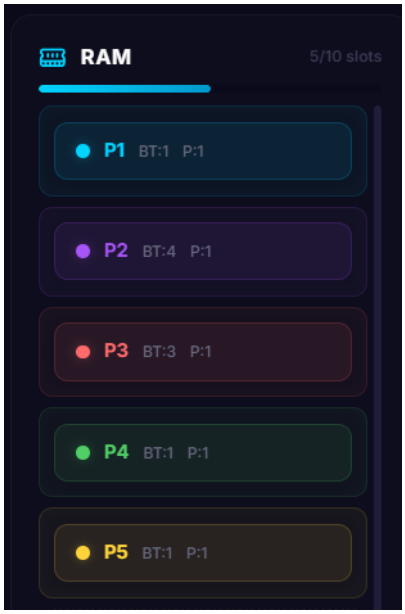

Figure 4: RAM, showing processes waiting and residing in memory during execution.

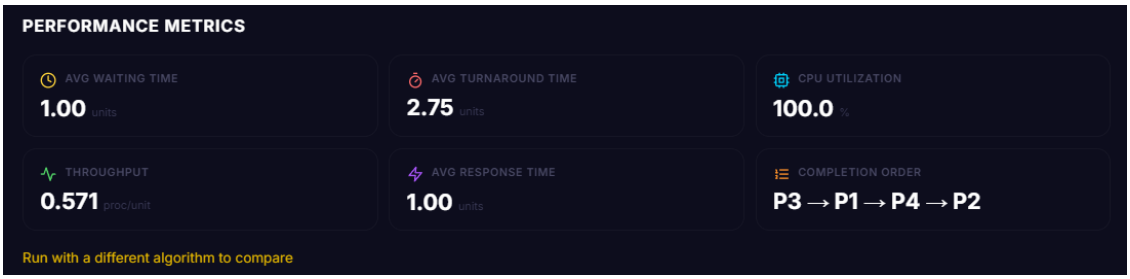

Figure 5: Performance Metrics After Completion, displaying final calculated values for the executed workload.

Figure 6: Adaptive Feedback Message, providing an algorithm recommendation based on the executed workload.

Figure 7: Priority Aging in Process Table, showing dynamically updated effective-priority values to prevent starvation.

10 Limitations

The current implementation is intentionally focused on CPU scheduling and educational clarity. It does not simulate actual disk I/O bursts, blocked-state transitions, multi-core execution, or context-switch overhead.

These limitations are reasonable for a semester project because the main goal is to explain core scheduling behavior clearly. Adding lower-level operating system details would make the system more realistic, but also much more complex and harder to demonstrate in a short viva.

11 Future Work

Possible extensions include:

- Adding disk I/O bursts and blocked-state handling.
- Supporting multi-core scheduling.
- Modeling context-switch overhead.
- Exporting results to PDF or CSV.
- Adding chart-based comparisons between algorithms.
- Saving and loading process sets.
- Adding more advanced workload classification for feedback.

12 Project Links

The source code and live deployment of this project are available at the following links:

- **Live Deployment:**
  `https://os-048-051.netlify.app/`
- **Umm-e-Habiba's GitHub:**
  `https://github.com/habiba-imran/Interactive-CPU-Scheduling--Simulator`
- **Zainab's GitHub:**
  `https://github.com/zainabidrees1234/CPU_Scheduler.git`

13 Conclusion and Learning Outcomes

This project demonstrates the complete life cycle of an interactive operati ng systems simulator: process modeling, algorithm selection, live visualization, metric evaluation, and adaptive feedback generation. It strengthens understanding of CPU scheduling trade-offs, starvation handling, responsiveness, fairness, and practical system design.

The implementation reinforces core computer science concepts such as modular design, state management, and algorithmic efficiency.

From a learning perspective, the most important lesson is that scheduling is not only about formulas. It is about understanding the consequences of policy decisions. This simulator makes those consequences visible, measurable, and comparable.

The adaptive feedback feature also adds value for demonstrations because it explains why a different algorithm may be more suitable for the current workload instead of simply listing metrics.